

MÓDULO 5 – ABSTRAÇÃO

Classes Abstratas & Interfaces

Como definir **contratos**: descrever o que uma classe deve fazer, sem dizer ainda como ela faz. A peça que faltava para amarrar herança e polimorfismo.

```
from abc import ABC
```

```
@abstractmethod
```

```
Protocol
```

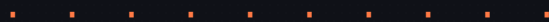
O QUE VEREMOS

Roteiro

BLOCO	TEMA
01	O que é abstração
02	O problema sem contrato
03	Classes abstratas (ABC)
04	Métodos abstratos
05	Abstrato + concreto na mesma classe

BLOCO	TEMA
06	Interfaces: o conceito
07	Interface em Python (ABC e Protocol)
08	Classe abstrata × interface
09	Contratos de comportamento
10	Exercícios e fechamento

Esta aula fecha o ciclo iniciado em herança e polimorfismo: agora vamos **obrigar** as subclasses a cumprir um conjunto de comportamentos.



01

Classes Abstratas

Modelar uma ideia genérica que existe para ser especializada — e nunca usada diretamente.

A IDEIA CENTRAL

Abstração: focar no essencial

Abstrair é separar **o que** uma coisa faz de **como** ela faz. Você descreve o comportamento esperado e esconde os detalhes da implementação.

Pense em "veículo": todo veículo **acelera** e **freia**. Mas **como** um carro elétrico acelera é diferente de uma bicicleta. O conceito "veículo" é uma **abstração**: ninguém dirige um "veículo genérico" — dirige um carro, uma moto, algo concreto.

Abstração no dia a dia

- ◆ Um **controle remoto**: você aperta "volume +", sem saber a eletrônica por trás.
- ◆ O **pedal do freio**: a ação é a mesma, o mecanismo muda entre carros.
- ◆ Uma **tomada**: define um formato (contrato); qualquer aparelho que o respeite funciona.

Em POO, a abstração vira código através de **classes abstratas** e **interfaces**: ferramentas para dizer "toda classe deste tipo precisa saber fazer X" sem definir o X agora.

POR QUE PRECISAMOS DISSO

Herança comum não garante nada

No polimorfismo, confiávamos que toda forma teria `area()`. Mas e se alguém esquecer de implementar?

```
class Forma:
    def area(self):
        print("área não definida")

class Quadrado(Forma):
    def __init__(self, lado):
        self.lado = lado
        # esqueceram de implementar area()!

q = Quadrado(5)
print(q.area())  # "área não definida"
```

● O programa **não quebra** — ele silenciosamente faz a coisa errada. O bug só aparece muito depois, talvez em produção.

Queríamos algo mais forte: uma forma de **impedir** que Quadrado exista enquanto não tiver um `area()` de verdade. Um **contrato obrigatório**.

A FERRAMENTA

Classe abstrata

Uma **classe abstrata** é uma classe que serve apenas de **molde** para outras. Ela:

- ◆ **Não pode ser instanciada** diretamente.
- ◆ Define métodos que as filhas são **obrigadas** a implementar.
- ◆ Existe para ser **herdada**, nunca usada sozinha.

Em Python, usamos o módulo abc (**Abstract Base Class**).

```
from abc import ABC, abstractmethod

class Forma(ABC):          # herda de ABC
    @abstractmethod
    def area(self):
        ...                # sem corpo
```

Dois ingredientes: herdar de ABC e marcar o método com o decorador `@abstractmethod`.

A GARANTIA EM AÇÃO

Python agora te protege

```
from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self): ...

# tentar instanciar a abstrata:
f = Forma()
```

```
# Python recusa na hora:
TypeError: Can't instantiate
abstract class Forma without an
implementation for abstract
method 'area'
```

✓ O erro acontece **imediatamente**, não escondido lá na frente. O contrato é verificado pelo próprio Python.

A mesma proteção vale para subclasses incompletas: se `Quadrado(Forma)` não implementar `area()`, tentar criar um `Quadrado()` também levanta `TypeError`.

CUMPRINDO O CONTRATO

A filha implementa, então funciona

```
from abc import ABC, abstractmethod
import math

class Forma(ABC):
    @abstractmethod
    def area(self): ...

class Quadrado(Forma):
    def __init__(self, lado):
        self.lado = lado
    def area(self):          # cumpre o contrato
        return self.lado ** 2

class Circulo(Forma):
    def __init__(self, raio):
        self.raio = raio
    def area(self):
        return math.pi * self.raio ** 2
```

```
>>> q = Quadrado(5)
>>> q.area()
25
>>> c = Circulo(2)
>>> c.area()
12.566...
```

Agora é **impossível** criar uma forma sem área. A abstração deixou de ser uma promessa e virou uma **regra imposta pela linguagem**.

PRATIQUE – CLASSE ABSTRATA

Exercício 1

1

Instrumentos musicais

Crie uma classe abstrata Instrumento (herdando de ABC) com um método abstrato tocar().

Crie Violao e Bateria, cada um implementando tocar() com uma mensagem própria.

PARA TESTAR

```
# deve funcionar:  
Violao().tocar()  
Bateria().tocar()  
  
# deve dar TypeError:  
Instrumento()
```

Confirme que instanciar Instrumento diretamente é recusado pelo Python, e que uma classe filha sem tocar() também não pode ser criada.

O MELHOR DOS DOIS MUNDOS

Métodos concretos na classe abstrata

Uma classe abstrata pode misturar métodos **abstratos** (obrigatórios nas filhas) com métodos **concretos** (já implementados e herdados normalmente).

```
class Funcionario(ABC):
    def __init__(self, nome):
        self.nome = nome

    @abstractmethod
    def calcular_salario(self): ... # cada um calcula

    def apresentar(self):           # concreto: comum a todos
        s = self.calcular_salario()
        print(f"{self.nome} recebe R${s}")
```

```
class Mensalista(Funcionario):
    def __init__(self, nome, salario):
        super().__init__(nome)
        self.salario = salario
    def calcular_salario(self):
        return self.salario

Mensalista("Ana", 3000).apresentar()
# Ana recebe R$3000
```

O método concreto apresentar() usa o abstrato calcular_salario() sem saber como ele foi feito. Abstração pura.

Exercício 2

2

Relatórios

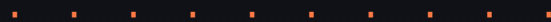
Crie uma classe abstrata Relatorio com:

- ◆ Um método abstrato gerar_conteudo().
- ◆ Um método concreto exportar() que imprime um cabeçalho fixo e depois chama gerar_conteudo().

Crie RelatorioVendas e RelatorioEstoque, cada um implementando gerar_conteudo() de forma diferente.

Chame exportar() nos dois e observe que o cabeçalho é o mesmo, mas o conteúdo muda.

O objetivo é exercitar a coexistência de um método obrigatório (abstrato) com um reaproveitado (concreto).



02

Interfaces

Contratos puros de comportamento: o que uma classe promete saber fazer, sem trazer nenhuma bagagem de implementação.

O QUE É UMA INTERFACE

Interface: só o contrato

Uma **interface** é uma lista de comportamentos que uma classe se compromete a ter — **sem nenhuma implementação**. É um contrato puro: "quem assinar isto precisa saber fazer A, B e C".

A analogia da **tomada**: a interface define o formato dos pinos. Não importa se o aparelho é um abajur ou uma TV — se o plugue encaixa, funciona. A interface garante a **compatibilidade**.

A ideia em uma frase

Classe abstrata diz "você **é** um tipo de mim".
Interface diz "você **sabe agir** como eu espero".

Foco no **comportamento**, não na linhagem.

JEITO 1 – ABC SÓ COM MÉTODOS ABSTRATOS

Interface com ABC

Python não tem a palavra-chave interface. Uma forma de criar uma é uma classe abstrata **sem nenhum método concreto** — só o contrato.

```
from abc import ABC, abstractmethod

class Notificavel(ABC):
    @abstractmethod
    def enviar(self, msg): ...

class Email(Notificavel):
    def enviar(self, msg):
        print(f"[Email] {msg}")

class SMS(Notificavel):
    def enviar(self, msg):
        print(f"[SMS] {msg}")
```

```
def avisar(canais, msg):
    for c in canais:
        c.enviar(msg)

avisar([Email(), SMS()],
        "Pedido aprovado")
# [Email] Pedido aprovado
# [SMS] Pedido aprovado
```

Qualquer classe que "assine" Notificavel é obrigada a ter enviar(). A função avisar confia nesse contrato.

JEITO 2 – PROTOCOL (DUCK TYPING ESTRUTURAL)

Interface com Protocol

Lembra do duck typing? O módulo `typing` oferece `Protocol`: define um contrato que a classe cumpre **sem precisar herdar** dele — basta ter os métodos certos.

```
from typing import Protocol

class Area(Protocol):
    def calcular_area(self) -> float: ...

# NÃO herda de Area:
class Sala:
    def calcular_area(self):
        return 20.0
```

```
def mostrar(x: Area):
    print(x.calcular_area())

mostrar(Sala())    # 20.0
# Sala cumpre o contrato
# só por TER o método
```

Protocol é o "se faz quack como um pato..." formalizado. Combina com o estilo flexível do Python e é muito usado em código moderno.

PRATIQUE — INTERFACE

Exercício 3

3

Formas de pagamento

Crie uma interface `MeioPagamento` (use ABC com um único método abstrato `pagar(valor)`).

Crie três classes que a implementam: `Pix`, `Cartao` e `Boleto`, cada uma com sua própria mensagem.

Escreva uma função `finalizar(meio, valor)` que receba qualquer `MeioPagamento` e chame `pagar(valor)` — sem usar `if` para checar o tipo.

Teste passando os três meios diferentes para a mesma função.

Repare como a interface garante que qualquer meio futuro (ex.: `CriptoMoeda`) funcionará na função sem alterá-la.

A DIFERENÇA QUE CAI EM PROVA

Classe abstrata × Interface

	CLASSE ABSTRATA	INTERFACE (CONTRATO PURO)
Pode ter código pronto?	Sim — mistura métodos concretos e abstratos.	Não — só assinaturas de métodos, sem corpo.
Pode ter atributos/estado?	Sim — pode ter <code>__init__</code> e atributos.	Geralmente não — foca só em comportamento.
Relação que expressa	"É UM" (linhagem) — Cachorro é um Animal.	"SABE FAZER" (capacidade) — sabe ser comparado.
Em Python	ABC com métodos concretos + abstratos.	ABC só abstrata, ou Protocol.

Regra prática: use **classe abstrata** quando há código comum para compartilhar entre as filhas; use **interface** quando só quer garantir que classes (mesmo não aparentadas) saibam executar certas ações.

POR QUE ISSO TUDO IMPORTA

Contratos de comportamento

Tanto a classe abstrata quanto a interface estabelecem um **contrato**: um acordo de que certos métodos **existirão**, com certo nome e finalidade.

Quem escreve uma função que depende do contrato pode confiar nele **sem conhecer as classes concretas**. Isso permite:

- ◆ **Trocar implementações** sem quebrar o resto do código.
- ◆ **Adicionar tipos novos** sem tocar nas funções existentes.
- ◆ **Detectar erros cedo**, quando o contrato é violado.

O contrato na prática

Contrato: "todo Meio sabe pagar()"



finalizar() confia no contrato



Pix, Cartao, Cripto... todos encaixam

A função nunca muda. O sistema cresce só adicionando classes que respeitam o contrato.

PRATIQUE — INTEGRANDO TUDO

Exercício 4 — Desafio

4

Sistema de exportação

Crie uma classe abstrata Exportador com:

- ◆ Atributo dados (recebido no construtor).
- ◆ Método abstrato formatar().
- ◆ Método concreto salvar() que imprime "Salvando..." e o resultado de formatar().

Implemente ExportadorJSON e ExportadorCSV, cada um formatando os dados à sua maneira.

Coloque vários exportadores numa lista e percorra chamando salvar() em cada um (polimorfismo, sem if de tipo).

Extra: tente criar um Exportador direto e confirme o TypeError.

Este exercício reúne abstração, método concreto + abstrato, herança, super() e polimorfismo num só lugar.

RESUMO

O que levamos

Abstração

Separar o **que** do **como**: descrever comportamento e esconder detalhes.

Classe abstrata

ABC + @abstractmethod. Não instancia; obriga as filhas a implementar. Pode ter código concreto.

Interface

Contrato puro de comportamento. Em Python: ABC sem código concreto ou Protocol.

Contrato

Garantia de que métodos existirão — código flexível, extensível e seguro.

Abstração é a tampa de tudo o que vimos: encapsulamento esconde dados, herança reaproveita, polimorfismo varia o comportamento — e a abstração **define as regras** que tornam tudo isso confiável.

MÓDULO 5 – CONCLUÍDO

Defina o contrato.

Deixe cada classe cumprir do seu jeito.

Com abstração, herança, encapsulamento e polimorfismo, você tem os quatro pilares da Orientação a Objetos.

4 exercícios

ABC + Protocol

contratos